Assignment 11.1

Name: Sunny Kolluri

HTNO: 2303A52444

Batch: 43

Task 1:

Prompt:

Generate a complete Python implementation of a Stack class.
The class should include the following methods:

- push(item) – to add an element to the stack
- pop () – to remove and return the top element
- peek () – to return the top element without removing it
- is_ empty () – to check whether the stack is empty

Requirements:

- Use a Python list internally to store elements
- Include proper Google-style docstrings for the class and all methods
- Add example usage at the bottom showing how the stack works
- Handle edge cases like popping from an empty stack

```python
#docstrings.
class Stack:
    """A simple implementation of a stack data structure."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack.

        Args:
            item: The item to be added to the stack.
        """
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack.

        Returns:
            The item at the top of the stack if it is not empty; otherwise, None.
        """
        if not self.is_empty():
            return self.items.pop()
        return None
```

Welcome | assignment 1 | Lab exam.py | Session 11.py | Assignment 9.1.py | Assign

Session 11.py > ...

```python
class Stack:

    def peek(self):
        """Return the item at the top of the stack without removing it.

        Returns:
            The item at the top of the stack if it is not empty; otherwise, None.
        """
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        """Check if the stack is empty.

        Returns:
            True if the stack is empty; otherwise, False.
        """
        return len(self.items) == 0
# Example usage:
stack = Stack()
stack.push(1)
stack.push(2)
print(stack.peek())  # Output: 2
print(stack.pop())   # Output: 2
print(stack.is_empty())  # Output: False
print(stack.pop())   # Output: 1
print(stack.is_empty())  # Output: True
```

```
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Script
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.ven
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.ven
 Assit/Session 11.py"
2
 Assit/Session 11.py"
2
2
2
False
1
True
```

The generated Stack class correctly satisfies the given task requirements.

1. All required methods are implemented:
    - Push () adds an element to the top of the stack.
    - Pop () removes and returns the top element.
    - Peek () returns the top element without removing it.
    - is_ empty () checks whether the stack is empty.
2. Internal Implementation:
    - The stack uses a Python list (self. items) to store elements, which is appropriate and efficient.
3. Docstrings:
    - The class and all methods include clear and meaningful docstrings.
    - Parameters and return values are properly described.
4. Edge Case Handling:
    - The pop () and peek () methods safely handle empty stack situations by returning None.
5. Example Usage:
    - The example demonstrates correct stack behaviour (LIFO – Last In, First Out)

Task 2:

Prompt:

Generate a complete Python implementation of a Queue class using Python lists. The class should follow the FIFO (First In, First Out) principle and include the methods enqueue(item), dequeue (), peek (), and size (). Include proper Google-style docstrings for the class and each method. Handle edge cases such as dequeue or peeking from an empty queue. Provide a short example usage demonstrating the queue operations.

```python
print(stack.is_empty())  # Output: True
#Generate a complete Python implementation of a Queue class using Python lists. The class should follow the FIFO (First
class Queue:
    """A simple implementation of a queue data structure using Python lists."""

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the end of the queue.

        Args:
            item: The item to be added to the queue.
        """
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue.

        Returns:
            The item at the front of the queue if it is not empty; otherwise, None.
        """
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def peek(self):
        """Return the item at the front of the queue without removing it.

        Returns:
            The item at the front of the queue if it is not empty; otherwise, None.
        """
        if not self.is_empty():
            return self.items[0]
        return None
```

```python
    def size(self):
        """Return the number of items in the queue.

        Returns:
            The number of items in the queue.
        """
        return len(self.items)

    def is_empty(self):
        """Check if the queue is empty.

        Returns:
            True if the queue is empty; otherwise, False.
        """
        return len(self.items) == 0
# Example usage:
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.peek())  # Output: 1
print(queue.size())  # Output: 2
print(queue.dequeue())  # Output: 1
print(queue.dequeue())  # Output: 2
print(queue.is_empty())  # Output: True
```

```
1
2
1
2
True
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- All required methods (enqueue, dequeue, peek, size, is_empty) are implemented.
- Follows **FIFO (First In First Out)** principle correctly.

- Docstrings are properly written.
- Empty queue case is handled safely.
- Example usage demonstrates correct behaviour

Task 3:

Prompt:

Generate a complete Python implementation of a Singly Linked List. Create a Node class and a LinkedList class. The LinkedList class should include insert(data) and display() methods. Add proper Google-style docstrings for all classes and methods. Ensure the implementation is functional and include a short example usage demonstrating insertion and display.

```python
#Generate a complete Python implementation of a Singly Linked List. Create a Node class and a LinkedList cla
class Node:
    """A node in a singly linked list."""

    def __init__(self, data):
        """Initialize a node with data and a reference to the next node.

        Args:
            data: The data stored in the node.
        """
        self.data = data
        self.next = None
class LinkedList:
    """A simple implementation of a singly linked list."""

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the linked list.

        Args:
            data: The data to be inserted into the linked list.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
```

```python
    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()  # for a new line after displaying the list
# Example usage:
linked_list = LinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.display()  # Output: 1 2 3
```

```
1 2 3
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Node and LinkedList classes are correctly defined.
- Insert () adds elements at the end properly.
- Display () prints elements correctly.
- Code structure is clean and readable.
- Docstrings are included.

Task 4:

Prompt:

Generate a complete Python implementation of a Binary Search Tree (BST). The class should include recursive insert(value) and in order traversal () methods. Ensure the insert method maintains BST properties. Add clear Google-style docstrings and provide example usage demonstrating insertion and in-order traversal.

```python
#Generate a complete Python implementation of a Binary Search Tree (BST). The class should include recursive ins
class TreeNode:
    """A node in a binary search tree."""

    def __init__(self, value):
        """Initialize a tree node with a value and references to left and right children.

        Args:
            value: The value stored in the tree node.
        """
        self.value = value
        self.left = None
        self.right = None
class BinarySearchTree:
    """A simple implementation of a binary search tree (BST)."""

    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, value):
        """Insert a new value into the BST while maintaining BST properties.

        Args:
            value: The value to be inserted into the BST.
        """
        if self.root is None:
            self.root = TreeNode(value)
        else:
            self._insert_recursive(self.root, value)
```

```python
    def _insert_recursive(self, node, value):
        """Helper method to insert a value recursively in the BST."""
        if value < node.value:
            if node.left is None:
                node.left = TreeNode(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = TreeNode(value)
            else:
                self._insert_recursive(node.right, value)

    def in_order_traversal(self):
        """Perform in-order traversal of the BST and return a list of values."""
        return self._in_order_recursive(self.root)

    def _in_order_recursive(self, node):
        """Helper method to perform in-order traversal recursively."""
        values = []
        if node:
            values.extend(self._in_order_recursive(node.left))
            values.append(node.value)
            values.extend(self._in_order_recursive(node.right))
        return values
# Example usage:
bst = BinarySearchTree()
bst.insert(5)
bst.insert(3)
bst.insert(7)
bst.insert(2)
bst.insert(4)
print(bst.in_order_traversal())  # Output: [2, 3, 4, 5, 7]
```

```
[2, 3, 4, 5, 7]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Recursive insertion maintains BST properties.
- In-order traversal correctly returns sorted elements.
- Helper recursive methods are properly used.
- Code is structured and documented clearly.
- Example output verifies correctness.

Task 5

Hash Table Prompt

Generate a Python implementation of a Hash Table with collision handling using chaining (linked lists). Include insert (key, value), search(key), and delete(key) methods. Add clear comments and Google-style docstrings explaining each method. Provide example usage to demonstrate insertion, searching, and deletion.

Session 11.py > ...

```python
class HashTable:
    def __init__(self, size=10):
        """Initialize the hash table with a specified size.

        Args:
            size: The number of buckets in the hash table (default is 10).
        """
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key.

        Args:
            key: The key to be hashed.

        Returns:
            An index corresponding to the bucket for the key.
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table.

        Args:
            key: The key to be inserted.
            value: The value associated with the key.
        """
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))
```

```python
Session 11.py > ...
class HashTable:
    def search(self, key):
        """Search for a value by its key in the hash table.

        Args:
            key: The key to be searched.

        Returns:
            The value associated with the key if found; otherwise, None.
        """
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def delete(self, key):
        """Delete a key-value pair from the hash table by its key.

        Args:
            key: The key to be deleted.

        Returns:
            True if the deletion was successful; otherwise, False.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False
# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
```

```
Alice
30
None
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Collision handling is correctly implemented using chaining.
- insert, search, and delete methods work properly.
- Existing keys are updated correctly.
- Docstrings and comments explain functionality clearly.
- Example usage demonstrates correct operations.

Task 6

Graph (Adjacency List) Prompt

Generate a Python implementation of a Graph using an adjacency list representation. Include methods add_ vertex(vertex), add_ edge (v1, v2), and display (). Add proper Google-style docstrings and provide example usage showing how vertices and edges are added and displayed.

```python
#Generate a Python implementation of a Graph using an adjacency list representation. Include m
class Graph:
    """A simple implementation of a graph using an adjacency list representation."""

    def __init__(self):
        """Initialize an empty graph."""
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph.

        Args:
            vertex: The vertex to be added to the graph.
        """
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, v1, v2):
        """Add an edge between two vertices in the graph.

        Args:
            v1: The first vertex.
            v2: The second vertex.

        Note:
            This method assumes that both vertices already exist in the graph.
        """
        if v1 in self.adjacency_list and v2 in self.adjacency_list:
            self.adjacency_list[v1].append(v2)
            self.adjacency_list[v2].append(v1)  # For undirected graph
```

```python
    def display(self):
        """Display the adjacency list of the graph."""
        for vertex, edges in self.adjacency_list.items():
            print(f"{vertex}: {edges}")
# Example usage:
graph = Graph()
graph.add_vertex("A")
graph.add_vertex("B")
graph.add_vertex("C")
graph.add_edge("A", "B")
graph.add_edge("B", "C")
graph.display()
```

```
A: ['B']
B: ['A', 'C']
C: ['B']
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Graph is correctly implemented using an adjacency list (dictionary).
- add_ vertex and add_ edge methods function properly.
- Undirected graph behaviour is correctly handled.
- Display () prints adjacency list clearly.
- Code includes proper documentation.

Task 7

Priority Queue Prompt

Generate a Python implementation of a Priority Queue using the built-in q module. Include methods enqueue (item, priority), dequeue (), and display ().

Add proper Google-style docstrings and example usage demonstrating priority-based removal (highest priority first).

```python
import queue
class PriorityQueue:
    """A simple implementation of a priority queue using the built-in queue module."""

    def __init__(self):
        """Initialize an empty priority queue."""
        self.pq = queue.PriorityQueue()

    def enqueue(self, item, priority):
        """Add an item to the priority queue with a specified priority.

        Args:
            item: The item to be added to the priority queue.
            priority: The priority level of the item (lower numbers indicate higher priority).
        """
        self.pq.put((priority, item))

    def dequeue(self):
        """Remove and return the item with the highest priority from the queue.

        Returns:
            The item with the highest priority if the queue is not empty; otherwise, None.
        """
        if not self.pq.empty():
            return self.pq.get()[1]  # Return only the item, not the priority
        return None

    def display(self):
        """Display all items in the priority queue in order of their priorities."""
        temp_list = []
        while not self.pq.empty():
            temp_list.append(self.pq.get())

        for priority, item in sorted(temp_list):
            print(f"Priority: {priority}, Item: {item}")
```

```python
    def display(self):
        """Display all items in the priority queue in order of their priorities."""
        temp_list = []
        while not self.pq.empty():
            temp_list.append(self.pq.get())

        for priority, item in sorted(temp_list):
            print(f"Priority: {priority}, Item: {item}")

        # Re-add items back to the priority queue
        for priority, item in temp_list:
            self.pq.put((priority, item))
# Example usage:
pq = PriorityQueue()
pq.enqueue("Task 1", priority=2)
pq.enqueue("Task 2", priority=1)
pq.display()
print(pq.dequeue())  # Output: Task 2 (highest priority)
print(pq.dequeue())  # Output: Task 1
print(pq.dequeue())  # Output: None (queue is empty)
```

```
Priority: 1, Item: Task 2
Priority: 2, Item: Task 1
Task 2
Task 1
None
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Priority queue is implemented using Python's built-in module.
- enqueue and dequeue correctly manage priority.
- Highest priority (lowest number) is removed first.
- Display method temporarily removes and restores elements properly.
- Edge cases are handled.

Task: 8

Deque Prompt

Generate a Python implementation of a double-ended queue using collections. Deque. Create a class Deque DS with methods to insert and remove elements from both front and rear. Add proper Google-style docstrings and example usage.

```python
from collections import deque
class DequeDS:
    """A simple implementation of a double-ended queue (deque) using collections.deque."""

    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()

    def insert_front(self, item):
        """Insert an item at the front of the deque.

        Args:
            item: The item to be inserted at the front of the deque.
        """
        self.deque.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear of the deque.

        Args:
            item: The item to be inserted at the rear of the deque.
        """
        self.deque.append(item)

    def remove_front(self):
        """Remove and return the item at the front of the deque.

        Returns:
            The item at the front of the deque if it is not empty; otherwise, None.
        """
        if self.deque:
            return self.deque.popleft()
        return None
```

```python
Session 11.py > ...
class DequeDS:

    def remove_front(self):
        """Remove and return the item at the front of the deque.

        Returns:
            The item at the front of the deque if it is not empty; otherwise, None.
        """
        if self.deque:
            return self.deque.popleft()
        return None

    def remove_rear(self):
        """Remove and return the item at the rear of the deque.

        Returns:
            The item at the rear of the deque if it is not empty; otherwise, None.
        """
        if self.deque:
            return self.deque.pop()
        return None

    def display(self):
        """Display all items in the deque."""
        print("Deque contents:", list(self.deque))
# Example usage:
deque_ds = DequeDS()
deque_ds.insert_rear("Item 1")
deque_ds.insert_rear("Item 2")
deque_ds.display()
deque_ds.insert_front("Item 0")
deque_ds.display()
print(deque_ds.remove_front())  # Output: Item 0
print(deque_ds.remove_rear())   # Output: Item 2
deque_ds.display()  # Output: Deque contents: ['Item 1']
```

```
Deque contents: ['Item 1', 'Item 2']
Deque contents: ['Item 0', 'Item 1', 'Item 2']
Item 0
Item 2
Deque contents: ['Item 1']
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Implemented correctly using collections deque.
- Supports insertion and removal from both ends.
- Methods handle empty deque safely.
- Display method shows contents clearly.
- Docstrings are properly written

Task 9

Campus Resource Management System Prompt

Generate a structured response for a Campus Resource Management System.

1. Select the most appropriate data structure (Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque) for each feature:
    - Student Attendance Tracking
    - Event Registration System
    - Library Book Borrowing
    - Bus Scheduling System
    - Cafeteria Order Queue
2. Present the answer in a table format:
   Feature → Chosen Data Structure → Justification (2–3 sentences each)
3. Implement one selected feature as a complete working Python program with proper comments and Google-style docstrings.
4. | Feature | Chosen Data Structure | Justification |
5. |---|---|---|
6. | Student Attendance Tracking | Stack | A stack is ideal for tracking attendance because it follows the Last-In-First-Out (LIFO) principle. New students are added to the top of the stack, and attendance can be recorded by removing students from the top. This ensures that the most recently added student is marked as present first. |
7. | Event Registration System | Queue | A queue is suitable for event registration because it follows the First-In-First-Out (FIFO) principle. Registrations are processed in the order they are received, ensuring fairness and consistency in handling registrations. |

8. | Library Book Borrowing | Stack | A stack is appropriate for library book borrowing because books are borrowed and returned in a LIFO manner. The most recently returned book is typically the next one to be borrowed, making a stack an intuitive choice for managing book availability. |
9. | Bus Scheduling System | Priority Queue | A priority queue is ideal for bus scheduling because different buses may have different priorities based on their routes or schedules. Higher-priority buses are scheduled first, ensuring efficient resource allocation and timely departures. |
10. | Cafeteria Order Queue | Queue | A queue is suitable for cafeteria orders because it ensures that orders are processed in the order they are received, maintaining fairness and preventing any order from being skipped or delayed unnecessarily. |

```python
class CafeteriaOrderQueue:
    """A simple implementation of a cafeteria order queue using a list."""

    def __init__(self):
        """Initialize an empty order queue."""
        self.orders = []

    def enqueue(self, order):
        """Add an order to the end of the queue.

        Args:
            order: The order to be added to the queue.
        """
        self.orders.append(order)

    def dequeue(self):
        """Remove and return the order at the front of the queue.

        Returns:
            The order at the front of the queue if it is not empty; otherwise, None.
        """
        if self.orders:
            return self.orders.pop(0)
        return None

    def display(self):
        """Display all orders in the queue."""
        print("Cafeteria Order Queue:", self.orders)
# Example usage:
```

```python
# Example usage:
cafeteria_queue = CafeteriaOrderQueue()
cafeteria_queue.enqueue("Order 1: Sandwich")
cafeteria_queue.enqueue("Order 2: Salad")
cafeteria_queue.enqueue("Order 3: Soup")
cafeteria_queue.display()  # Output: Cafeteria Order Queue: ['Order 1: Sandwich', 'Order 2: Salad', 'Order 3: Soup']
print(cafeteria_queue.dequeue())  # Output: Order 1: Sandwich
cafeteria_queue.display()  # Output: Cafeteria Order Queue: ['Order 2: Salad', 'Order 3: Soup']
```

```
Deque contents: ['Item 1', 'Item 2']
Deque contents: ['Item 0', 'Item 1', 'Item 2']
Item 0
Item 2
Deque contents: ['Item 1']
Cafeteria Order Queue: ['Order 1: Sandwich', 'Order 2: Salad', 'Order 3: Soup']
Order 1: Sandwich
Cafeteria Order Queue: ['Order 2: Salad', 'Order 3: Soup']
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Appropriate data structure (Queue) was selected for Cafeteria Order feature.
- FIFO behaviour correctly models real-world serving order.
- Implementation is simple and functional.
- Code is properly documented.
- Example usage demonstrates correct behaviour

Task 10

Smart E-Commerce Platform Prompt

Generate a structured response for a Smart E-Commerce System.

1. Select the most appropriate data structure (Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque) for each feature:
    - Shopping Cart Management
    - Order Processing System
    - Top-Selling Products Tracker

- Product Search Engine
- Delivery Route Planning

2. Present the answer in a table format:
   Feature → Chosen Data Structure → Justification (2–3 sentences each)
3. Implement one selected feature as a complete working Python program with proper comments and Google-style docstrings.

```python
class ShoppingCart:
    """A simple implementation of a shopping cart using a list."""

    def __init__(self):
        """Initialize an empty shopping cart."""
        self.cart = []

    def add_item(self, item):
        """Add an item to the shopping cart.

        Args:
            item: The item to be added to the cart.
        """
        self.cart.append(item)

    def remove_item(self, item):
        """Remove an item from the shopping cart if it exists.

        Args:
            item: The item to be removed from the cart.

        Returns:
            True if the item was removed; otherwise, False.
        """
        if item in self.cart:
            self.cart.remove(item)
            return True
        return False

    def display_cart(self):
        """Display all items in the shopping cart."""
        print("Shopping Cart:", self.cart)
```

```python
# Example usage:
shopping_cart = ShoppingCart()
shopping_cart.add_item("Laptop")
shopping_cart.add_item("Headphones")
shopping_cart.add_item("Mouse")
shopping_cart.display_cart()  # Output: Shopping Cart: ['Laptop', 'Headphones', '
shopping_cart.remove_item("Headphones")
shopping_cart.display_cart()  # Output: Shopping Cart: ['Laptop', 'Mouse']
```

```
Deque contents: [ Item 1 ]
Cafeteria Order Queue: ['Order 1: Sandwich', 'Order 2: Salad', 'Order 3: Soup']
Order 1: Sandwich
Cafeteria Order Queue: ['Order 2: Salad', 'Order 3: Soup']
Shopping Cart: ['Laptop', 'Headphones', 'Mouse']
Shopping Cart: ['Laptop', 'Mouse']
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Shopping cart implemented using list (dynamic and simple).
- Add and remove operations work correctly.
- Code includes proper documentation.
- Example usage verifies functionality.
- Suitable data structure chosen for the feature.